QuantLens App Business Plan

An Android App for Hong Kong Stock Event Research and Personal Notes

Mobile Smart Application Development
Team 7 | English edition: 20 October 2026

Team Leader: 段博铧
Team Members: 段博铧, 胡与同, 张胡文康, 陈钰龙

1. App Overview

QuantLens is a Hong Kong stock event research app for people learning about finance. We want users to find important events about companies they follow, open the original information, write down their understanding and return to it later, all on a phone. The first release is planned as an online pilot for invited users, with stock and event updates provided over the network and watchlists and research notes stored on the device.

A typical research task

Consider a student following a company when its annual results are announced. The student reads a summary, opens the original announcement and notes what needs further investigation. For example, did a change in profit come mainly from the core business or from a one-off item? When new information becomes available, the student can return to the same note and add an observation, without having to find the announcement and scattered notes again.

This is the task QuantLens is designed around: keeping a company event together with the research notes that develop around it. Finding information is one part of the task; preserving how the user understood it is another.

Unique Selling Proposition

QuantLens focuses on the relationship between events, sources and personal notes. Each event will show its source and publication time. A note will link directly to the relevant event, keeping the original interpretation and later observations available together. We want beginners to spend less time organizing materials and more time reading and thinking.

Clarity matters as much as the feature list. Users should be able to see which date the data belongs to, where a summary came from and which statements are their own judgments. These details will appear where users read and write.

First-release scope

The first release will cover invited-user login, stock search, watchlists, event reading, research notes and review. The client will be developed in Android Studio using Kotlin and Jetpack Compose. We will first complete one research workflow, then use pilot feedback to decide what to add. Cross-device synchronization, push notifications and advanced research tools will be evaluated in later versions.

The project is currently at the product-design and development-planning stage. No Android project has been created. The screens, technical arrangements and pilot activities described here are plans; implementation and test evidence will be added when available.

Product position: help finance learners verify Hong Kong stock events and keep research notes they can return to.

2. Users and Market

Target User Persona

Our initial users are university students and other finance learners interested in Hong Kong stocks and company events. They understand basic stock-market concepts, mainly use a phone to access information and want to develop their own research habits. This persona guides the initial scope and will be revised through interviews.

Their tasks may be brief: checking a new announcement between classes, verifying a detail before a discussion or reviewing the week's notes at the weekend. Sources need to be easy to find and notes easy to save, with little setup required.

Existing tools and our entry point

Established products already offer charts, market information and news. QuantLens will organize its features around a beginner's event-research task. We will compare how well users can complete that task rather than compete on feature count.

Tool or approach	Relevant existing capabilities	QuantLens design focus
TradingView	Charts, alerts, screening and historical replay [1].	Keep an event's source linked to the research note written at the time.
Tiger Trade	Watchlist-related news, paper trading and AI features [2].	Connect event reading with the user's own interpretation.
HKEXnews with a separate note-taking tool	Issuer-announcement search, with notes organized elsewhere [3].	Reduce switching between finding the original source, writing and reviewing notes.

This comparison explains our entry point. Whether the proposed workflow suits our users better still needs to be tested. Participants will check a source and save a note while we record the steps, time taken and points of confusion. The comparison does not imply that competing products lack all related features.

Starting on campus

We will begin with classmates, finance study groups and relevant campus societies. These channels make it practical to arrange interviews, explain the trial and invite the same people back after changes. The first online pilot will aim to recruit 10 to 20 users to identify problems and explore willingness to continue using the app before expanding.

We are particularly interested in whether participants return voluntarily to read events and organize notes. A download or a positive comment during a demonstration does not establish continued use.

3. Product Plan

Users will open a stock from their watchlist and view its related events. After reading an event, they can open the original source or save a research note linked to it. Returning to the event should bring the existing note back into view and allow further observations.

Module	First-release scope	User benefit
Invited-user login	Trial accounts issued by the team, with loading, failure and logout states.	A clear way to enter the pilot and access the correct personal records.
Watchlists and stock details	Search by name or stock code, manage followed companies, and view a simple price trend with its data date.	Keep information about followed companies in one place.
Event reading	Events linked to stocks, showing category, summary, publication time and an original-source link.	Understand what happened and continue checking the evidence.
Research notes	Record an interpretation, evidence, questions and points to follow up.	Keep an understanding alongside the event it concerns.
Review and export	Keep the original note, add observations and export records locally.	Revisit earlier judgments and back up personal records.

Reading and writing on a phone

The bottom navigation will contain Watchlist, Events, Notes and Settings. The main screen will prioritize followed companies. Event details will make the time and source prominent, with a note entry point near the reading content. Simple price charts will leave space for reading and writing.

Price changes will use signs or text as well as colour. Longer content will support system font scaling. Saving a note will produce clear feedback, and returning to a list should preserve its position. A network interruption, an empty event list and a failed save will have different messages so users can tell what happened.

Intelligence where it helps

The first release will link events to followed stocks and organize them into categories such as results, dividends and share repurchases. Summaries will help users decide whether to read the original document. Important figures, dates and company names should remain traceable to that source. If automatic summarization is introduced later, factual consistency will be checked before it is opened to pilot users.

The notes area will distinguish source information from the user's interpretation. Review will place later information alongside the original note so users can revise their understanding. Price changes will be observations, not evidence on their own that an event caused a particular market movement.

4. Development and Operation

Client and data services

The mobile client and data service will be developed separately. The service will organize stock and event information and expose a consistent API. The client will handle presentation, interaction and personal records. Watchlists, notes and essential event caches will be stored locally, with information refreshed when connectivity is available. Android's offline-first architecture provides an established basis for this approach [4].

Development and debugging will use Android Studio. The client stack will include Kotlin, Jetpack Compose, ViewModel and Room, with a single MainActivity hosting the screens. Navigation Compose is the proposed navigation approach and will be confirmed before coding. Dependency versions will be checked for compatibility when the project is created.

The UI will display content and collect actions. ViewModel will manage screen state, and Repository will provide a common interface to network and local data. Before implementation, the team will agree on stock codes, event identifiers, account ownership and time fields so modules can be integrated and problems traced.

Information sources

Company disclosures will retain their original sources. HKEXnews can provide an entry point for checking announcements [3]. Market and event data will be refreshed periodically, with publication time and the date of market data shown separately. If an update fails, the last available content will remain visible with a status message, allowing users to judge its relevance to the task.

Coverage, pricing and display permissions will be checked before a data service is integrated. We will display permitted content and preserve attribution. Service credentials will stay on the server, and the client will access the required endpoints over HTTPS. Pilot access scope and request frequency will be controlled separately.

Personal records and pilot delivery

Personal records in the first release will mainly stay on the device, be read according to account ownership and have a local export option. Login identifies the user and controls service access; cloud synchronization is outside this initial arrangement. Logout or account switching will clear the session and old screen state so that the previous account's records are not displayed. Users will also be informed about the effects of uninstalling, changing devices and upgrading.

Once the online build is ready, invited users will receive a signed APK [5]. Before testing begins, the team will check external network access, saving and rereading notes after restart, offline reading, original-source links and upgrades. Service issues and feedback will then be tracked centrally, with an owner assigned to each item.

Keeping the scope manageable

We will first complete the search, event and note workflow, then connect it to update services. Every proposed addition should identify the step it improves and the development and maintenance work it adds. Broader stock coverage and automation will follow a stable core workflow.

5. Login, Navigation and Saving Notes

Login involves more than a form. The first release will use trial accounts issued by the team, verified by the server over HTTPS. Signed-out, validating, failed-login and expired-session states will have clear feedback. The server will still enforce access permissions; hiding a screen does not provide authentication. Self-service registration, password recovery and payments are outside the first release.

Entering a screen and returning

Action	Screen behaviour	Implementation and acceptance focus
Successful login	Open Watchlist and remove the login flow.	Pressing Back must not reopen the login screen.
Watchlist to stock to event	Locate content using stockId and eventId.	Pass minimal identifiers and load details from the data layer [6].
Event to editor to save	Return to the event after a successful save.	Write to the database before returning; preserve input if saving fails.
Back while editing	Ask how to handle unsaved changes.	Offer save and return, discard, or continue editing; keep toolbar and system Back behaviour consistent.
Logout	End the session and open Login.	Clear business-screen history and account-related restored state.

In-app navigation will use navigate to open a destination, popBackStack to return and popUpTo to clear history where appropriate [7]. Screens will share one Activity rather than use startActivity for internal navigation. Toolbar Back, system Back and bottom-navigation behaviour will be checked separately.

Saving must do more than show a message

After a note is saved, the detail screen and notes list should show its new content from the same data source. Room will persist records, while ViewModel will manage screen state. Temporary input and list position may be restored with rememberSaveable or SavedStateHandle where appropriate [8]. Passwords will not be stored as restorable screen state.

If a one-time saved result is needed, a noteId may be returned and cleared after consumption [9]. That message provides feedback; the database remains the basis for the content. Kotlin's return ends a function and does not perform screen navigation.

6. Marketing and Validation

Marketing Channels

Initial recruitment will use small-scale invitations. Each invitation will describe a concrete task, such as reading a company event and saving a research note, so participants know what they are being asked to do. We want to hear about difficulties early and change the product in response.

Channel	Approach	What we will observe
Classmates	Demonstrate the core workflow and invite participants to complete a task.	Whether they can independently find a source and save a note.
Finance study groups or societies	Share a case and trial instructions with the organizer's permission.	Whether participants have similar research needs.
Participant referrals	Ask willing participants to recommend the trial to classmates voluntarily.	New-user participation and subsequent use.

All four members will recruit participants and collect feedback using the same invitation and task script. We will record invitations, participation and task completion by channel to compare recruitment effort and results. The first round will rely on direct outreach, with no paid advertising planned.

Validation Channels

We plan to start with 5 interviews about participants' most recent stock-research activity: which tools they used, how they verified information and how they kept their views. We will then ask 5 users to try the prototype and observe points of confusion. Once the online build is ready, we will invite 10 to 20 users to the pilot. These numbers describe planned work; findings will only be reported after the activities take place.

The task will remain consistent: log in with a trial account, find an event for a specified company, check its source, save a linked research note, then return and reopen it. When comparing source-checking and note-taking workflows, we will use events of similar difficulty and alternate the order in which tools are tried. Login will be timed separately so authentication does not distort the comparison.

How we will judge usefulness

We will first observe whether users can complete the task independently, then record completion time and where they get stuck. During the pilot, we will also check whether participants return voluntarily within the stated observation period. Results will include participant counts, observation periods and test conditions to keep the small sample interpretable.

Working source links and complete note persistence are basic requirements. A faster task time will not count as an improvement if the user skips source verification or loses a record.

7. Business Model and Costs

Start with a free pilot

The first release will be free for invited users. We need to establish whether people want to keep using it and whether the team can maintain the data and service at an affordable level before considering paid features.

Later options may include multi-device synchronization, additional note-management tools and more advanced event organization. The basic reading and note-taking experience comes first. Any pricing will depend on feedback, willingness to pay and actual service costs. Payments are outside the first-release development scope.

Main costs

Operating expenses will mainly cover servers and a domain, data services and essential maintenance tools. Data may be charged by subscription or usage and will be tracked according to the provider's terms. The team will also record maintenance effort to understand how work changes as users or coverage increase.

The budget will be confirmed after quotations and data scope are known. If expenses exceed it, we will first adjust pilot size, update frequency or coverage before deciding whether to invest more. These choices must still allow users to complete the core research task.

Financial Key Metrics

During the pilot, we will record actual monthly operating expenditure and compare it with monthly active trial users to calculate an average service cost. An active trial user is someone who views at least one event or saves at least one note in that month. No average will be calculated when there are no active trial users. Server and data expenses will be tracked separately to avoid double counting.

If paid validation begins later, we will track paid conversion and revenue per paying user. Revenue remaining after variable service costs can then contribute towards fixed expenses. We will not forecast revenue scale at this stage; actual costs and usage will guide the decision to continue investing.

Ongoing considerations

Continued updates depend on a reliable, permitted data supply. Personal records matter to long-term use, so backup and upgrade checks will accompany releases. Each development round will review scope and prioritize the core workflow.

8. Team and Implementation Plan

Each of the four members owns a main module and shares research, testing and documentation. The initial plan assigns approximately one quarter of the workload to each person, including coordination within the team leader's allocation. Tasks will be rebalanced against actual difficulty and effort.

Member	Main responsibilities
段博铧 - Team Leader	App navigation, sessions and integration; product scope, consolidation of the project plan and submission coordination.
胡与同	Login interface, stock search, watchlists and stock details; competitor research, prototypes and module testing.
张胡文康	Event lists and details, source links and network states; API integration, interviews and source verification.
陈钰龙	Note editing, Room storage, review and export; saving and return behaviour, pilot metrics and cost tracking.

Each member will test their module and take part in cross-review. The initial pairs are 段博铧 with 胡与同, and 张胡文康 with 陈钰龙. Account and data interfaces will be agreed together before implementation is divided. Server-side work will be estimated separately so that unplanned responsibilities do not accumulate with the team leader.

Development stages

Stage	Main deliverables
First phase	Business plan, README, course repository and presentation materials for the scheduled report on 21 October.
Core functionality	Compose project, login interface, navigation, watchlists and research notes, with saving and return behaviour verified.
Online pilot	Real authentication and data services; account-isolation, offline, installation and upgrade checks before inviting users.

Later dates will be set against the course schedule and actual progress. Every task will identify the screen or API, key logic, acceptance method and owner. Early reports will use flow diagrams to explain the design. Actual screens, source-code locations and test records will be added after development; prototypes will be identified as prototypes.

Acceptance and workload evidence

Key checks will cover successful and failed login, opening and returning from details, rereading a saved note, offline caches and account isolation after logout. Contributions will be evidenced through task records, actual commits, cross-reviews and test materials, rather than line counts or commit counts alone.

Code explanations will focus on three areas: authentication state, the navigation stack and note persistence. Each will connect the screen action to a data change and visible feedback. The complete source will remain part of the project, with the submission format checked against course requirements.

9. Current Issues, Solutions and Future Development

The current issues concern design and preparation for delivery. There is no running Android version yet, so these gaps are not presented as reproduced software defects.

Current issue	Planned response	Evidence of completion
No Android project or device evidence yet.	Build the core workflow, then add screenshots, source code and test records.	A signed installable build and reproducible operations.
Authentication and data services have not been selected.	Confirm permissions and costs; start with a small stock universe and trial accounts.	Accessible APIs, working authentication and verifiable sources.
User needs and continued use remain untested.	Conduct interviews, prototype sessions and an invited pilot using the same task.	Original research records, participant counts and observation periods.

Future development

The immediate goal is reliable login, reading, note-taking and return behaviour. After the pilot, we will consider broader event coverage and improvements to search, export and accessibility. Cross-device synchronization and reminders will be evaluated after privacy and service costs are understood. Automatic summaries will follow a fact-checking process, with traceability to original sources as a condition for release.

Each progress report will describe what has actually been completed, the remaining problems, planned responses and next steps. Independent task completion, reliable records and sustainable operation will guide further investment and expansion.

References

[1] TradingView: Official Feature Overview
https://www.tradingview.com/features/

[2] Tiger Trade: Official App Updates
https://www.itigerup.com/en/about/app/update

[3] HKEXnews: Hong Kong Exchanges and Clearing
https://www.hkexnews.hk/index.htm

[4] Android Developers: Build an Offline-First App
https://developer.android.com/topic/architecture/data-layer/offline-first

[5] Android Developers: Sign Your App
https://developer.android.com/studio/publish/app-signing

[6] Android Developers: Navigation
https://developer.android.com/guide/navigation

[7] Android Developers: The Navigation Back Stack
https://developer.android.com/guide/navigation/backstack

[8] Android Developers: Save UI State in Compose
https://developer.android.com/develop/ui/compose/state-saving

[9] Android Developers: Interact Programmatically with the Navigation Component
https://developer.android.com/guide/navigation/use-graph/programmatic

Sources accessed on 19 October 2026.